Adaptive Controller Result Guide

Plant: **SinglePole** | Test: **test_1** | Mode: **mock**

Item	Recorded value
Source JSON	batch_tuning_results.json
Artifact	single_pole.json
Pipeline completed	true
Verification success	true
Best tuning round	3
Generated	2026-09-19T15:27:36+02:00

This guide separates four kinds of information: agent responses, tuning-round results, final numerical verification, and notebook packaging. The agents propose or describe decisions. Python performs the controller design, simulations, metric calculations, best-round selection and JSON/PDF writing.

1. Run flow and responsibility

Stage	What happens	Source
Designer	Chooses Backstepping or SMC from the plant structure.	Mock/LLM agent
Round 0	Runs the initial controller parameters.	Python simulation
Tuning rounds	Agent proposes gains; every proposal is simulated and scored.	Agent + Python
Best selection	Chooses the lowest score for the selected objectives.	Python
Verification	Runs the selected best parameters again on the final path.	Python simulation
Reporter	Creates a short abstract from the final outcome.	Mock/LLM agent
JSON/PDF	Organizes and saves all fields.	Notebook / this generator

2. Applied test

Field	Value
States	[theta, omega]
Inputs	[tau]
Outputs	[theta]
Initial state	[0.05, 0]
References	[{"expr": "0.0", "output": "theta"}]
Uncertainty	[]
Disturbance	[]
Simulation time	10
Solver step	0.001

3. Agent messages

The mock run records one Designer call, one Tuner call for each tuning round, and one Reporter call. Responses are seeded and repeatable. The numerical metrics that follow are calculated from simulations.

Design call 1

Response field	Value
method	backstepping
reasoning	MOCK Designer: selected backstepping from the plant structure. Python performs the real controller design and simulation.
notes_limitations	Mock agent output for testing, not an LLM recommendation.

Synthetic usage: input 588, output 91, total 679.

Tuner call 1

Response field	Value
reasoning	MOCK Tuner: seeded random gain adjustment for exercising the tuning loop. Improvements are decided by real simulation metrics.
diagnosed_symptom	slow_transient
c_gains	[1.9849046, 3.9698091]

Synthetic usage: input 549, output 138, total 687.

Tuner call 2

Response field	Value
reasoning	MOCK Tuner: seeded random gain adjustment for exercising the tuning loop. Improvements are decided by real simulation metrics.
diagnosed_symptom	slow_transient
c_gains	[2.2891669, 4.5783337]

Synthetic usage: input 464, output 117, total 581.

Tuner call 3

Response field	Value
reasoning	MOCK Tuner: seeded random gain adjustment for exercising the tuning loop. Improvements are decided by real simulation metrics.
diagnosed_symptom	slow_transient
c_gains	[2.3428196, 4.6856391]

Synthetic usage: input 961, output 84, total 1045.

Tuner call 4

Response field	Value
reasoning	MOCK Tuner: seeded random gain adjustment for exercising the tuning loop. Improvements are decided by real simulation metrics.
diagnosed_symptom	slow_transient

Response field	Value
c_gains	[2.1785789, 4.3571578]

Synthetic usage: input 440, output 187, total 627.

Reporter call 1

Response field	Value
abstract	MOCK Report Writer. SinglePole uses Backstepping. run verdict PASS; reference tracking 100.0% (worst output, steady state); tuning ran 4 round(s), target not fully met. The reported outcome comes from the numerical simulation; agent responses and token usage are simulated.

Synthetic usage: input 819, output 208, total 1027.

4. Best parameters and best-round metrics

best_params contains the controller parameters from the tuning round with the lowest internal objective score. This run selected only control_effort, so the ranking is based on control_rms relative to round 0. The objective weight is 3; with one objective, changing that single weight does not change the ranking.

Parameter	Best value
c_gains	[2.3428196, 4.6856391]
Gamma	25
kappa	5
k2	1
k3	1
k4	1
sigma_W	0.1
tau	0.05
N	25
width	1.5
rbf_spread	1
rbf_normalize	meanstd
use_filtered_error	false
lambda_I	0.5

Selected round: **3**. best_metrics is the simulation result recorded when this candidate was tested inside the tuning loop.

Metric	Best-round value	Meaning
numerically_healthy	true	All states, outputs and controls stayed finite.
success	true	True only when finite, bounded and mse_target all pass.
success_checks.finite	true	No NaN or infinity occurred.
success_checks.bounded	true	Output stayed below 1000 times the effective task scale and |u| stayed below 1e12.
success_checks.mse_target	true	Every output's steady RMS fraction was at or below success_target_frac.

Metric	Best-round value	Meaning
success_reason	empty (passed)	The first failed hard check; empty means all checks passed.
success_target_frac	0.02	Allowed steady RMS error as a fraction of task scale; 0.02 means 2%.
tracking_pct.full	[80.897357]	Tracking percentage over the complete simulation.
tracking_pct.steady	[100]	Tracking percentage over the final 20%.
tracking_pct.transient	[57.286114]	Tracking percentage over the first 20%.
tracking_mse.full	[9.122774e-05]	Mean squared tracking error over the full simulation.
tracking_mse.steady	[1.520157e-23]	Mean squared tracking error over the final 20%.
tracking_mse.transient	[0.00045611901]	Mean squared tracking error over the first 20%.
task_scale	[0.05]	RMS error of an output frozen at its initial value. Here it equals the initial 0.05 rad error.
task_trivial	[false]	True only when the initial output is already effectively at its reference.
tracking_pct_headline	100	Worst steady tracking percentage across controlled outputs.
tracking_pct_mean	100	Mean steady tracking percentage across controlled outputs.
transient_rms	[0.021356943]	RMS tracking error over the first 20%; radians for this pole.
steady_rms	[3.898919e-12]	RMS tracking error over the final 20%; radians for this pole.
steady_rms_frac	[7.797838e-11]	steady_rms divided by effective task scale; used by mse_target.
overshoot_frac	[5.000000e+07]	Maximum absolute error divided by reference amplitude with a 1e-9 floor.
settling_time	7.86	First time after which error always remains inside the 2% reference-amplitude band.
settling_time_reached	true	Whether that settling condition was reached within the horizon.
control_rms	[0.028178279]	RMS control torque over the simulation; the selected tuning objective.
control_max	[0.2223873]	Largest absolute control torque.
control_rate_max	[0.7015343]	Largest absolute finite-difference rate |du/dt|.
chattering_tv_per_sec	[0.022240955]	Total variation of the control per second; mainly an SMC diagnostic.

5. Exact metric formulas

Quantity	Formula
Tracking error	e[k] = y[k] - reference[k]
Window MSE	MSE = mean(e[k]^2) within full, first 20%, or last 20%
Window RMS	RMS = sqrt(MSE)
Task scale	S = RMS(y[0] - reference[k]); here S = 0.05 rad
Tracking percentage	100 * max(0, 1 - RMS / S)
Steady RMS fraction	steady_rms / effective_task_scale
Success target	steady_rms_frac <= success_target_frac for every output
Control RMS	sqrt(mean(u[k]^2))
Control maximum	max(abs(u[k]))
Control rate maximum	max(abs((u[k+1] - u[k]) / dt))

Quantity	Formula
Control variation	sum(abs(u[k+1] - u[k])) / simulation_duration

The JSON key mse_target is historical naming: the implemented hard check uses steady normalized RMS, not raw MSE.

Zero-reference warning: overshoot_frac divides maximum absolute error by max reference amplitude, with a 1e-9 floor. For this zero reference, 0.05 / 1e-9 = 50,000,000. That large value is a normalization artifact, not physical overshoot. The settling band also uses 2% of the floored reference amplitude, only 2e-11 rad here, so the reported 7.86 s is much stricter than a practical task-scale settling band.

6. Tuning history

History is the initial design plus the result after each tuning proposal. Round 0 uses default gains. Each later row records the gains actually simulated, what changed, the simulation verdict and the selected objective value.

Round	c_gains	Changed	Pass	Target met	Steady tracking %	Objective
0	[2, 4]	{}	true	false	100	[[Control effort, 0.0288 RMS(u), n/a]]
1	[1.9849046, 3.9698091]	{"c_gains": [[2.0, 4.0], [1.98490457, 3.96980914]]}	true	false	100	[[Control effort, 0.0288 RMS(u), n/a]]
2	[2.2891669, 4.5783337]	{"c_gains": [[1.98490457, 3.96980914], [2.28916687, 4.57833375]]}	true	false	100	[[Control effort, 0.0282 RMS(u), n/a]]
3	[2.3428196, 4.6856391]	{"c_gains": [[2.28916687, 4.57833375], [2.34281956, 4.68563912]]}	true	false	100	[[Control effort, 0.0282 RMS(u), n/a]]
4	[2.1785789, 4.3571578]	{"c_gains": [[2.34281956, 4.68563912], [2.17857891, 4.35715782]]}	true	false	100	[[Control effort, 0.0284 RMS(u), n/a]]

success and met_target are different. success is the numerical/verification verdict. met_target controls tuner early stopping. Since this run selected only control_effort and no tracking objective, the implementation deliberately keeps met_target false and runs all configured tuning rounds, even when the tracking success check passes.

Round 0

Reasoning: (initial design, before tuning)

Parameters: {"Gamma": 25, "N": 25, "c_gains": [2.0, 4.0], "k2": 1, "k3": 1, "k4": 1, "kappa": 5, "lambda_I": 0.5, "rbf_normalize": "meanstd", "rbf_spread": 1.0, "sigma_W": 0.1, "tau": 0.05, "use_filtered_error": false, "width": 1.5}

Changed: none

```
RUN VERDICT: PASS
REFERENCE TRACKING: 100.0% (worst output, steady-state) | 100.0% mean | 79.6% full-window

NUMERICAL HEALTH: OK

TRACKING
- transient RMS (first 20%): [0.0228]
- steady-state RMS (last 20%): [0.0000]
- steady-state RMS as a fraction of the task scale (same basis as your target and the mse_target check
above): [0.0000]
- overshoot (max deviation, as a fraction of reference amplitude): [50000000.0000]
- settling time (first time error enters and stays within a 2% band): 8.286s

CONTROL EFFORT
- RMS(u): [0.0288]
- max|u|: [0.1976]
- max|du/dt|: [0.45]
```

Round 1

Reasoning: MOCK Tuner: seeded random gain adjustment for exercising the tuning loop. Improvements are decided by real simulation metrics.

Parameters: {"Gamma": 25, "N": 25, "c_gains": [1.98490457, 3.96980914], "k2": 1, "k3": 1, "k4": 1, "kappa": 5, "lambda_I": 0.5, "rbf_normalize": "meanstd", "rbf_spread": 1.0, "sigma_W": 0.1, "tau": 0.05, "use_filtered_error": false, "width": 1.5}

Changed: {"c_gains": [[2.0, 4.0], [1.98490457, 3.96980914]]}

```
RUN VERDICT: PASS
REFERENCE TRACKING: 100.0% (worst output, steady-state) | 100.0% mean | 79.5% full-window

NUMERICAL HEALTH: OK

TRACKING
- transient RMS (first 20%): [0.0229]
- steady-state RMS (last 20%): [0.0000]
- steady-state RMS as a fraction of the task scale (same basis as your target and the mse_target check
above): [0.0000]
- overshoot (max deviation, as a fraction of reference amplitude): [50000000.0000]
- settling time (first time error enters and stays within a 2% band): 8.282s

CONTROL EFFORT
- RMS(u): [0.0288]
- max|u|: [0.1966]
- max|du/dt|: [0.44]
```

Round 2

Reasoning: MOCK Tuner: seeded random gain adjustment for exercising the tuning loop. Improvements are decided by real simulation metrics.

Parameters: {"Gamma": 25, "N": 25, "c_gains": [2.28916687, 4.57833375], "k2": 1, "k3": 1, "k4": 1, "kappa": 5, "lambda_I": 0.5, "rbf_normalize": "meanstd", "rbf_spread": 1.0, "sigma_W": 0.1, "tau": 0.05, "use_filtered_error": false, "width": 1.5}

Changed: {"c_gains": [[1.98490457, 3.96980914], [2.28916687, 4.57833375]]}

```
RUN VERDICT: PASS
REFERENCE TRACKING: 100.0% (worst output, steady-state) | 100.0% mean | 80.7% full-window

NUMERICAL HEALTH: OK

TRACKING
- transient RMS (first 20%): [0.0216]
- steady-state RMS (last 20%): [0.0000]
- steady-state RMS as a fraction of the task scale (same basis as your target and the mse_target check
above): [0.0000]
- overshoot (max deviation, as a fraction of reference amplitude): [50000000.0000]
- settling time (first time error enters and stays within a 2% band): 7.954s

CONTROL EFFORT
- RMS(u): [0.0282]
- max|u|: [0.2182]
- max|du/dt|: [0.66]
```

Round 3

Reasoning: MOCK Tuner: seeded random gain adjustment for exercising the tuning loop. Improvements are decided by real simulation metrics.

Parameters: {"Gamma": 25, "N": 25, "c_gains": [2.34281956, 4.68563912], "k2": 1, "k3": 1, "k4": 1, "kappa": 5, "lambda_I": 0.5, "rbf_normalize": "meanstd", "rbf_spread": 1.0, "sigma_W": 0.1, "tau": 0.05, "use_filtered_error": false, "width": 1.5}

Changed: {"c_gains": [[2.28916687, 4.57833375], [2.34281956, 4.68563912]]}

```
RUN VERDICT: PASS
REFERENCE TRACKING: 100.0% (worst output, steady-state) | 100.0% mean | 80.9% full-window

NUMERICAL HEALTH: OK

TRACKING
- transient RMS (first 20%): [0.0214]
- steady-state RMS (last 20%): [0.0000]
- steady-state RMS as a fraction of the task scale (same basis as your target and the mse_target check
above): [0.0000]
- overshoot (max deviation, as a fraction of reference amplitude): [50000000.0000]
- settling time (first time error enters and stays within a 2% band): 7.860s

CONTROL EFFORT
- RMS(u): [0.0282]
- max|u|: [0.2224]
- max|du/dt|: [0.70]
```

Round 4

Reasoning: MOCK Tuner: seeded random gain adjustment for exercising the tuning loop. Improvements are decided by real simulation metrics.

Parameters: {"Gamma": 25, "N": 25, "c_gains": [2.17857891, 4.35715782], "k2": 1, "k3": 1, "k4": 1, "kappa": 5, "lambda_I": 0.5, "rbf_normalize": "meanstd", "rbf_spread": 1.0, "sigma_W": 0.1, "tau": 0.05, "use_filtered_error": false, "width": 1.5}

Changed: {"c_gains": [[2.34281956, 4.68563912], [2.17857891, 4.35715782]]}

```
RUN VERDICT: PASS
REFERENCE TRACKING: 100.0% (worst output, steady-state) | 100.0% mean | 80.3% full-window

NUMERICAL HEALTH: OK

TRACKING
- transient RMS (first 20%): [0.0220]
- steady-state RMS (last 20%): [0.0000]
- steady-state RMS as a fraction of the task scale (same basis as your target and the mse_target check
above): [0.0000]
- overshoot (max deviation, as a fraction of reference amplitude): [50000000.0000]
- settling time (first time error enters and stays within a 2% band): 8.132s

CONTROL EFFORT
- RMS(u): [0.0284]
- max|u|: [0.2100]
- max|du/dt|: [0.57]
```

7. Token usage

Field	Value	Meaning
input_tokens	3821	Total synthetic prompt tokens.
output_tokens	825	Total synthetic response tokens.
total_tokens	4646	Input plus output tokens.
cached_input_tokens	0	Synthetic cached subset; zero here.
simulated	true	True means these are test values, not tokenizer measurements.
mock_seed	42	Seed controlling repeatable mock output.
price_usd_per_token	0.001	Synthetic benchmark price.
cost_usd	4.646	Synthetic total_tokens times synthetic price.
actual_cost_usd	0	Actual API charge; zero for mock mode.

Agent	Round	Label	Input	Output	Total	Detail
design	0	Mock method selection	588	91	679	ok
tuner	1	Tuning, round 1	549	138	687	target not yet met
tuner	2	Tuning, round 2	464	117	581	target not yet met
tuner	3	Tuning, round 3	961	84	1045	target not yet met
tuner	4	Tuning, round 4	440	187	627	target not yet met

8. Final verification

After choosing best_params, the pipeline runs those parameters once more through the final simulation path. verification contains that final result. best_metrics and verification therefore refer to two executions of the same selected parameters. In this deterministic run their numerical values match. Verification additionally includes mse_target_from_tuner=true, which records that the tuner target was used.

Metric	Verification value	Meaning
numerically_healthy	true	All states, outputs and controls stayed finite.

Metric	Verification value	Meaning
success	true	True only when finite, bounded and mse_target all pass.
success_checks.finite	true	No NaN or infinity occurred.
success_checks.bounded	true	Output stayed below 1000 times the effective task scale and |u| stayed below 1e12.
success_checks.mse_target	true	Every output's steady RMS fraction was at or below success_target_frac.
success_reason	empty (passed)	The first failed hard check; empty means all checks passed.
success_target_frac	0.02	Allowed steady RMS error as a fraction of task scale; 0.02 means 2%.
tracking_pct.full	[80.897357]	Tracking percentage over the complete simulation.
tracking_pct.steady	[100]	Tracking percentage over the final 20%.
tracking_pct.transient	[57.286114]	Tracking percentage over the first 20%.
tracking_mse.full	[9.122774e-05]	Mean squared tracking error over the full simulation.
tracking_mse.steady	[1.520157e-23]	Mean squared tracking error over the final 20%.
tracking_mse.transient	[0.00045611901]	Mean squared tracking error over the first 20%.
task_scale	[0.05]	RMS error of an output frozen at its initial value. Here it equals the initial 0.05 rad error.
task_trivial	[false]	True only when the initial output is already effectively at its reference.
tracking_pct_headline	100	Worst steady tracking percentage across controlled outputs.
tracking_pct_mean	100	Mean steady tracking percentage across controlled outputs.
transient_rms	[0.021356943]	RMS tracking error over the first 20%; radians for this pole.
steady_rms	[3.898919e-12]	RMS tracking error over the final 20%; radians for this pole.
steady_rms_frac	[7.797838e-11]	steady_rms divided by effective task scale; used by mse_target.
overshoot_frac	[5.000000e+07]	Maximum absolute error divided by reference amplitude with a 1e-9 floor.
settling_time	7.86	First time after which error always remains inside the 2% reference-amplitude band.
settling_time_reached	true	Whether that settling condition was reached within the horizon.
control_rms	[0.028178279]	RMS control torque over the simulation; the selected tuning objective.
control_max	[0.2223873]	Largest absolute control torque.
control_rate_max	[0.7015343]	Largest absolute finite-difference rate |du/dt|.
chattering_tv_per_sec	[0.022240955]	Total variation of the control per second; mainly an SMC diagnostic.
mse_target_from_tuner	true	The final success threshold came from the tuning configuration.

pipeline_completed=true means method selection, controller construction and final simulation completed and time-series data existed. verification.success=true is the separate statement that the final run passed finite, bounded and steady-error checks.

9. Summary field

The summary is built by Python from the selected method, symbolic controller, stability proof, final parameters and verification metrics. The Reporter does not create this full section. Its short abstract is stored in the reporter entry inside mock_agent_messages; the current compact notebook JSON does not copy that abstract to a root-level field.